1. What is a Function in Python?

A **function** is a block of reusable code that performs a specific task.
Functions help in:

- Reducing code repetition
- Improving readability
- Making programs modular and easy to debug
- *Write once, use many times.*

---

2. Types of Functions in Python

1. Built-in Functions

These are predefined functions provided by Python.

Examples:

```
print("Hello")
len([1, 2, 3])
type(10)
sum([1, 2, 3])
```

---

2. User-Defined Functions

Functions created by the programmer using the `def` keyword.

---

3. Syntax of a Python Function

```
def function_name(parameters):
    statement(s)
    return value
```

Explanation

- `def` → keyword to define a function
- `function_name` → name of the function
- `parameters` → input values (optional)
- `return` → sends output back (optional)

---

5. Simple Function Example

```
def greet():
    print("Hello, Welcome to Python")
```

Function call:

```
greet()
```

---

6. Function with Parameters

```
def add(a, b):
    print(a + b)

add(10, 20)
```

✔ `a` and `b` are parameters
✔ Values passed during call are arguments

---

7. Function with Return Value

```
def square(num):
    return num * num

result = square(5)
print(result)
```

👉 `return` sends value back to the caller

---

8. Types of Function Arguments

---

1. Positional Arguments

Arguments passed in order.

```
def sub(a, b):
    print(a - b)

sub(10, 5)
```

---

2. Keyword Arguments

Arguments passed using parameter names.

```
sub(b=5, a=10)
```

```
def sub(a, b):
print("Subtraction:", a - b)

sub(b=5, a=10)
```

---

3. Default Arguments

Default values are used if no argument is given.

```
def greet(name="Student"):
    print("Hello", name)

greet()
greet("Amit")
```

---

4. Variable-Length Arguments

Variable-length arguments allow a function to accept **any number of arguments**.

There are two types:

- `*args` → Non-keyword (positional) arguments
- `**kwargs` → Keyword arguments

1️⃣ `*args` (Non-Keyword Arguments)

`*args` allows you to pass multiple positional arguments.
It stores them in a **tuple**.

✅ Example:

```
def add(*args):
    total = 0
    for num in args:
        total += num
    print("Sum:", total)

add(10, 20, 30, 40)
```

🔹 Output:

```
Sum: 100
```

`**kwargs` (Keyword Arguments)

`**kwargs` allows you to pass multiple keyword arguments.
It stores them in a **dictionary**.

✅ Example:

```
def student_info(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)

student_info(name="Amit", age=40, city="Pune")
```

🔹 Output:

```
name : Amit
age : 40
city : Pune
```

---

9. Function Without Return (Void Function)

```
def show_message():
    print("This function returns nothing")
```

---

10. Recursive Functions

What is Recursion?

Recursion is a technique where a function **calls itself** to solve a smaller part of a problem.

Instead of using loops, the function repeats itself until a stopping condition is met.

🔷 Two Important Parts of Recursion

1️⃣ **Base Case** → Condition to stop recursion
2️⃣ **Recursive Case** → Function calls itself

If there is no base case ❌ → infinite recursion (error)

🔷 Example 1: Factorial Using Recursion

```
def print_numbers(n):
    if n == 0:        # Base Case
        return
    print(n)
    print_numbers(n - 1)    # Recursive Call

print_numbers(5)
```

Output:

```
5
4
3
2
1
```

📌 Factorial Formula:

$$n! = n \times (n - 1)!$$

Example:
$5! = 5 \times 4 \times 3 \times 2 \times 1$

💻 Python Code:

```
def factorial(n):
    if n == 1:           # Base Case
        return 1
    else:
        return n * factorial(n - 1)    # Recursive Case

print(factorial(5))
```

✅ Output:

```
120
```

🔷 How It Works (Step-by-Step)

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
```

```
= 5 * 4 * 3 * 2 * 1
= 120
```

When `n == 1`, recursion stops.

---

11. Lambda (Anonymous) Functions

Short, one-line functions without a name.

A **lambda function** is an **anonymous (nameless) function** written in a single line.

```
Syntax:
lambda arguments: expression
```

- Can take any number of arguments
- Has only one expression
- Automatically returns the result
- Does NOT use `return` keyword

```
square = lambda x: x * x
print(square(4))
```

Lambda with Multiple Arguments

```
add = lambda a, b: a + b

print(add(10, 20))
```

Lambda with `filter()`

`filter()` selects elements based on condition.

```
numbers = [1, 2, 3, 4, 5, 6]

even = list(filter(lambda x: x % 2 == 0, numbers))

print(even)
```

Used when:

- Function is small
- Required for short operations

---

12. Scope of Variables in Functions

1. Local Variable

Defined inside a function.

```
def test():
    x = 10
    print(x)
```

---

2. Global Variable

Defined outside the function.

```
x = 20

def show():
    print(x)
```

---

13. Function with List Example

```
def find_sum(numbers):
    return sum(numbers)

print(find_sum([10, 20, 30]))
```